

Challenge 2 IoT

Mattia Brianti 10773859
Alex Hathaway 10829946

1 CQ1

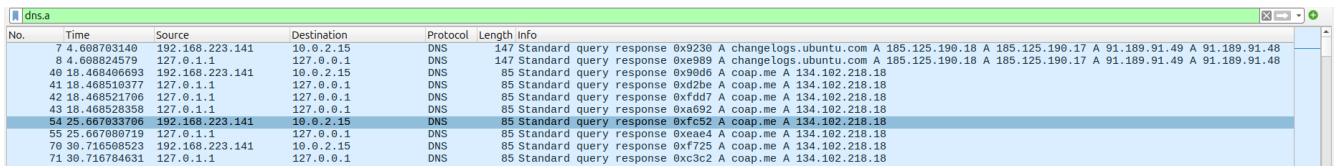
1.1 CQ1a

Question: Find all NON Confirmable DELETE requests directed to the coap.me server that received a successful response.

Answer: MID = 30800

Explanation:

To target the correct traffic, we first identified the IP address for coap.me by analyzing DNS queries in the capture. We did it using the dns.a command, finding these results:



No.	Time	Source	Destination	Protocol	Length	Info
7	4.608703140	192.168.223.141	10.0.2.15	DNS	147	Standard query response 0x9230 A changelogs.ubuntu.com A 185.125.190.18 A 185.125.190.17 A 91.189.91.49 A 91.189.91.48
8	4.608824579	127.0.1.1	127.0.0.1	DNS	147	Standard query response 0xe989 A changelogs.ubuntu.com A 185.125.190.18 A 185.125.190.17 A 91.189.91.49 A 91.189.91.48
40	18.468406693	192.168.223.141	10.0.2.15	DNS	85	Standard query response 0x90d6 A coap.me A 134.102.218.18
41	18.468519377	127.0.1.1	127.0.0.1	DNS	85	Standard query response 0xd2be A coap.me A 134.102.218.18
42	18.468521706	127.0.1.1	127.0.0.1	DNS	85	Standard query response 0xfdd7 A coap.me A 134.102.218.18
43	18.468528358	127.0.1.1	127.0.0.1	DNS	85	Standard query response 0xa692 A coap.me A 134.102.218.18
54	25.667033706	192.168.223.141	10.0.2.15	DNS	85	Standard query response 0xfc52 A coap.me A 134.102.218.18
55	25.667088719	127.0.1.1	127.0.0.1	DNS	85	Standard query response 0xeaa4 A coap.me A 134.102.218.18
76	30.716598523	192.168.223.141	10.0.2.15	DNS	85	Standard query response 0xf725 A coap.me A 134.102.218.18
71	30.716784631	127.0.1.1	127.0.0.1	DNS	85	Standard query response 0xc3c2 A coap.me A 134.102.218.18

Figure 1: DNS resolution

The result highlighted that the coap.me server had 134.102.218.18 as IP address. Once we discovered it we wrote the filters to find all the DELETE requests that were sent to the server. The filter contained the following:

1. coap.code == 4 for getting all the DELETE messages
2. coap.type == 1 for getting all NON CONFIRMABLE messages
3. ip.dst == 134.102.218.18 for getting all the messages directed to the coap.me server

All the packets being NON CONFIRMABLE messages and being a DELETE request are the ones in the image below, where we highlighted their MIDs and respective token:

```

alexh@alexh-ThinkPad-T14-Gen-4:~/Desktop/Internet Of Things/2026/Challenge2_26$
tshark -r A.pcapng -Y 'coap.type == 1 && coap.code == 4 && ip.dst == 134.102.218
.18' -T fields -e coap.mid -e coap.token > CQ1/requests.txt
alexh@alexh-ThinkPad-T14-Gen-4:~/Desktop/Internet Of Things/2026/Challenge2_26$
cat CQ1/requests.txt
33264      7b9376a6a9484b8b
53878      030bda63ff31592a
63041      f35bbf03a5ff38cc
38052      2301e4de4156749f
57936      e93dbbcb6a598a7d
25814      8b6f78cbe076f916
1059       b651a74c2a74aecd
32298      7359ba9493b39e78
24716      e0864998dd738206
38480      57f987c179a32d78
32242      7801af5ea0b348f0
30212      252ef436b7efa3ea
11787      21a587c5a0ae8a6e
22836      7f09e3677ebbc42
63788      07f20b6ad047d749
47220      1e6083b696fb3231
62535      737051f79ba1e4d2
49932      452a18979a587044
30800      880f30d8b543d6f7
21028      75912d3b51bc5016

```

Figure 2: DELETE requests

After finding this we'll need to look for successful responses. We define a "successful response" as a response containing the proper success code for that specific request. For a DELETE operation, the proper success code is 2.02 Deleted (Code 66). The filter we'll apply is "coap.code == 66 && ip.src == 134.102.218.18", which will highlight all the successful delete responses sent by the coap.me server. From the results we extract only the tokens. The packets found in this case are:

```

alexh@alexh-ThinkPad-T14-Gen-4:~/Desktop/Internet Of Things/2026/Challenge2_26$
tshark -r A.pcapng -Y "coap.code == 66 && ip.src == 134.102.218.18" -T fields -e
coap.token > CQ1/success_responses.txt
alexh@alexh-ThinkPad-T14-Gen-4:~/Desktop/Internet Of Things/2026/Challenge2_26$
cat CQ1/success_responses.txt
3186f7113ca5aba1
b78aa821b9c65d66
3ae775dfbd67941f
880f30d8b543d6f7

```

Figure 3: Successful responses

The answer to the question CQ1a can be found by joining the two sets (requests and successful responses) using the token of the messages. This is done because the messages exchanged are NON CONFIRMABLE which means we can't look at the MID of a message, but we must look for the same token to find a request-response exchange. The results show that only 1 request received a successful response. The MID of this request is 30800.

```
alexh@alexh-ThinkPad-T14-Gen-4:~/Desktop/Internet Of Things/2026/Challenge2_26$
grep -F -f CQ1/success_responses.txt CQ1/requests.txt | sort -u > CQ1/matching_r
equests.txt
alexh@alexh-ThinkPad-T14-Gen-4:~/Desktop/Internet Of Things/2026/Challenge2_26$
cat CQ1/matching_requests.txt
30800 880f30d8b543d6f7
```

Figure 4: Matching results

1.2 CQ1b

Question: *How many of the requests found in point CQ1a) actually obtained the desired outcome?*

Answer: 0

Explanation: In this case, for the term "desired outcome" we refer to a DELETE request that successfully deleted the specified resource. From the previous answer we know there is only one request that received a successful response from the server. Our goal now will be to verify if the resource indicated by the request having MID = 30800 has been successfully deleted. The first thing to do is to understand what resource the request wanted to delete. To do so we apply this filter: "coap.mid == 30800" and output the uri_path. This is the obtained result:

```
alexh@alexh-ThinkPad-T14-Gen-4:~/Desktop/Internet Of Things/2026/Challenge2_26$
tshark -r A.pcapng -Y "coap.mid == 30800" -T fields -e coap.opt.uri_path
validate
```

Figure 5: DELETE request resource

The result highlights that the DELETE request wanted to delete the "validate" resource. To see if the resource was effectively removed, we'll analyze the traffic and look if GET requests to the same resource received a successful response (2.05 Content). To search for this we apply this filter: "coap.opt.uri_path == "validate" || ip.src == 134.102.218.18" which looks for messages having as uri_path "validate" and messages that are sent by the coap.me server. The results are the ones below:

coap.opt.uri_path == "validate" ip.src == 134.102.218.18					
No.	Time	Source	Destination	Protocol	Length Info
10284	376.169621369	134.102.218.18	10.0.2.15	CoAP	69 NON, MID:43428, 2.01 Created, TKN:1c b6 09 83 83 44 88 76, /large-create
10360	381.188243323	134.102.218.18	10.0.2.15	CoAP	79 NON, MID:38570, 2.05 Content (text/plain), TKN:eb c3 58 83 5a c8 0d 90, /separate
10370	382.220199106	134.102.218.18	10.0.2.15	CoAP	95 NON, MID:43391, 2.01 Created (text/plain), TKN:b8 49 41 c3 e0 fe 7c 31, /sink
10464	390.117729715	134.102.218.18	10.0.2.15	CoAP	73 ACK, MID:6336, 2.05 Content (text/plain), TKN:b0 80 76 73 87 b1 a6 a0, /multi-format
10465	390.118012028	134.102.218.18	10.0.2.15	CoAP	73 ACK, MID:6336, 2.05 Content (text/plain), TKN:b0 80 76 73 87 b1 a6 a0, /multi-format
10547	393.244459980	134.102.218.18	10.0.2.15	CoAP	73 NON, MID:26930, 2.05 Content (text/plain), TKN:e7 1f c5 d5 f5 93 68 50, /multi-format
10608	396.197550502	134.102.218.18	10.0.2.15	CoAP	111 NON, MID:5190, 2.05 Content, TKN:63 26 76 ba a9 df 16 42, /location1
10664	412.206492295	134.102.218.18	10.0.2.15	CoAP	92 ACK, MID:57841, 2.05 Content (text/plain), TKN:47 03 95 ec 0b cc da e5, /weird33
10991	420.227835660	134.102.218.18	10.0.2.15	CoAP	92 ACK, MID:1004, 2.05 Content (text/plain), TKN:c4 43 63 52 60 9a 7a bc, /weird44
11060	427.390474017	134.102.218.18	10.0.2.15	CoAP	83 ACK, MID:47286, 4.05 Method Not Allowed (text/plain), TKN:af 6a 2e 21 b8 cb 9c 8e, /location1
11075	428.150151063	10.0.2.15	134.102.218.18	CoAP	65 NON, MID:30800, DELETE, TKN:88 0f 30 d8 b5 43 d6 f7, /validate
11078	428.332370619	134.102.218.18	10.0.2.15	CoAP	67 NON, MID:26010, 2.02 Deleted (text/plain), TKN:88 0f 30 d8 b5 43 d6 f7, /validate
11090	428.625992081	134.102.218.18	10.0.2.15	CoAP	83 ACK, MID:6924, 4.05 Method Not Allowed (text/plain), TKN:13 fa 8e d9 8e a5 5c a9, /weird333
11222	436.779775363	134.102.218.18	10.0.2.15	CoAP	83 ACK, MID:2950, 4.05 Method Not Allowed (text/plain), TKN:81 cd c7 a1 db b0 7c dc, /location-query
11363	445.398432889	10.0.2.15	134.102.218.18	CoAP	65 NON, MID:5096, GET, TKN:38 f2 48 20 7e f5 0a 87, /validate
11364	445.408027040	134.102.218.18	10.0.2.15	CoAP	71 NON, MID:11392, 2.05 Content (text/plain), TKN:32 f2 48 20 7e f5 0a 87, /validate
11485	453.192316733	134.102.218.18	10.0.2.15	CoAP	83 ACK, MID:51824, 4.05 Method Not Allowed (text/plain), TKN:9a 13 87 6b 8c 11 7f 7d, /separate
11533	457.389744585	134.102.218.18	10.0.2.15	CoAP	83 NON, MID:54485, 4.05 Method Not Allowed (text/plain), TKN:e0 d6 7e d4 2e 5b 26 6d, /location1
11626	457.239712858	134.102.218.18	10.0.2.15	CoAP	83 ACK, MID:18924, 4.05 Method Not Allowed (text/plain), TKN:28 03 67 09 d8 84 40 c7, /3
11850	466.398594798	134.102.218.18	10.0.2.15	CoAP	83 NON, MID:16232, 4.05 Method Not Allowed (text/plain), TKN:1e 5f ec cc 9c c5 3e 7d, /weird
11924	468.283404348	134.102.218.18	10.0.2.15	CoAP	83 NON, MID:1208, 4.05 Method Not Allowed (text/plain), TKN:75 91 2d 3b 51 bc 50 16, /3
11943	469.266557539	134.102.218.18	10.0.2.15	CoAP	83 NON, MID:5157, 4.05 Method Not Allowed (text/plain), TKN:85 9e 5c 02 0c 53 57 8f, /hello

Figure 6: Proof that the validate resource has not been deleted

From the darker-blue messages we clearly see that the DELETE request having MID = 30800 receives a successful response (2.02 Deleted), but afterwards when a GET request having MID = 5096, request the resource the response from the server is successful (2.05 Content), which means the desired outcome wasn't reached.

2 CQ2

Question: *How many CoAP resources in the local server received the same number of unique unsuccessful Confirmable POST and PUT requests?*

Answer: 1

Explanation:

To answer this question it is important to highlight that a response to a CONFIRMABLE message uses always the same MID of the original request. This helps us solving the question, because we can look just at the MIDs and not at the tokens. The first filter we used is "coap.code >= 128 && ip.src == 127.0.0.1" and searches for:

- coap.code >= 128: all response codes higher than 4, which indicate client and server errors
- ip.src == 127.0.0.1: all the messages coming from the local server

Below are shown some of the packets resulting from this first filtering part:

```
alexh@alexh-ThinkPad-T14-Gen-4:~/Desktop/Internet Of Things/2026/Challenge2_26$  
tshark -r A.pcapng -Y "coap.code >= 128 && ip.src == 127.0.0.1" -T fields -e coa  
p.mid | sort -u > CQ2/errors.txt  
alexh@alexh-ThinkPad-T14-Gen-4:~/Desktop/Internet Of Things/2026/Challenge2_26$  
cat CQ2/errors.txt  
10179  
10465  
10466  
10680  
1120  
11242  
11409  
12  
13  
1330  
14  
15  
15990  
17  
18  
19  
20  
21  
21082
```

Figure 7: Error messages

After doing this, we separate the POST and PUT requests that are made to the server. The two filters will differ only in the code field and will be: "coap.type == 0 && coap.code == x && ip.dst == 127.0.0.1":

1. coap.code == x: if x = 2 it filters POST requests, instead if x = 3 it filters PUT requests;
2. coap.type == 0 filters for only CONFIRMABLE messages
3. ip.dst == 127.0.0.1 is for getting all the messages directed to the local server

The results for the POST requests are:

```

alexh@alexh-ThinkPad-T14-Gen-4:~/Desktop/Internet Of Things/2026/Challenge2_26$
tshark -r A.pcapng -Y "coap.type == 0 && coap.code == 2 && ip.dst == 127.0.0.1"
-T fields -e coap.mid -e coap.opt.uri_path > CQ2/post_req.txt
alexh@alexh-ThinkPad-T14-Gen-4:~/Desktop/Internet Of Things/2026/Challenge2_26$
cat CQ2/post_req.txt
64080    test
21336    dining_room,temperature
53552    dining_room,temperature
11409    dining_room,temperature
25749    hello_world
6295     test
40958    dining_room
42780    basic
11242    basic
33536    main_door
21082    living_room,temperature
40534    living_room
40964    hello_post
40965    hello_post
10465    dining_room,temperature
10466    dining_room,temperature
26063    hello_world
58468    hello_post
9853     dining_room,door
10179    dining_room,door
29078    hello_world
57914    hello_world

```

Figure 8: POST requests messages

While the results for the PUT requests are the following:

```

alexh@alexh-ThinkPad-T14-Gen-4:~/Desktop/Internet Of Things/2026/Challenge2_26$
tshark -r A.pcapng -Y "coap.type == 0 && coap.code == 3 && ip.dst == 127.0.0.1"
-T fields -e coap.mid -e coap.opt.uri_path > CQ2/put_req.txt
alexh@alexh-ThinkPad-T14-Gen-4:~/Desktop/Internet Of Things/2026/Challenge2_26$
cat CQ2/put_req.txt
1330     main_door
48174    dining_room,temperature
50477    basic
33760    dining_room,door
6326     living_room,door
30514    main_door
10680    dining_room
10181    dining_room,door
37991    dining_room,door
40939    hello_post
40207    dining_room,temperature

```

Figure 9: PUT requests messages

Then we have to look, using the MIDs, for POST and PUT requests messages inside the list of errors we filtered before. To do so we'll use the grep command to match the MIDs we have in the errors with the ones we have in the PUT and POST requests. For the POST requests, these are the results:

```
alexh@alexh-ThinkPad-T14-Gen-4:~/Desktop/Internet Of Things/2026/Challenge2_26$  
grep -F -f CQ2/errors.txt CQ2/post_req.txt | sort -u > CQ2/failed_post.txt  
alexh@alexh-ThinkPad-T14-Gen-4:~/Desktop/Internet Of Things/2026/Challenge2_26$  
cat CQ2/failed_post.txt  
10179 dining_room,door  
10465 dining_room,temperature  
10466 dining_room,temperature  
11242 basic  
11409 dining_room,temperature  
21082 living_room,temperature  
21336 dining_room,temperature  
25749 hello_world  
26063 hello_world  
29078 hello_world  
33536 main_door  
40534 living_room  
40958 dining_room  
40964 hello_post  
40965 hello_post  
42780 basic  
53552 dining_room,temperature  
57914 hello_world  
58468 hello_post  
6295 test  
64080 test  
9853 dining_room,door
```

Figure 10: POST requests messages that failed

While for the PUT requests we have the following results:

```
alexh@alexh-ThinkPad-T14-Gen-4:~/Desktop/Internet Of Things/2026/Challenge2_26$
grep -F -f CQ2/errors.txt CQ2/put_req.txt | sort -u > CQ2/failed_put.txt
alexh@alexh-ThinkPad-T14-Gen-4:~/Desktop/Internet Of Things/2026/Challenge2_26$
cat CQ2/failed_put.txt
10181 dining_room,door
10680 dining_room
1330 main_door
30514 main_door
33760 dining_room,door
37991 dining_room,door
40207 dining_room,temperature
40939 hello_post
48174 dining_room,temperature
50477 basic
6326 living_room,door
```

Figure 11: PUT requests messages that failed

Once we've got all the POST and PUT failed requests, we have to count how many times each resource appears. These are the results:

```
alexh@alexh-ThinkPad-T14-Gen-4:~/Desktop/Internet Of Things/2026/Challenge2_26$
awk '{print $2}' CQ2/failed_post.txt | sort | uniq -c > CQ2/post_counts.txt
alexh@alexh-ThinkPad-T14-Gen-4:~/Desktop/Internet Of Things/2026/Challenge2_26$
cat CQ2/post_counts.txt
  2 basic
  1 dining_room
  2 dining_room,door
  5 dining_room,temperature
  3 hello_post
  4 hello_world
  1 living_room
  1 living_room,temperature
  1 main_door
  2 test
```

Figure 12: Number of times a resource appears in the failed POST requests

```
alexh@alexh-ThinkPad-T14-Gen-4:~/Desktop/Internet Of Things/2026/Challenge2_26$
awk '{print $2}' CQ2/failed_put.txt | sort | uniq -c > CQ2/put_counts.txt
alexh@alexh-ThinkPad-T14-Gen-4:~/Desktop/Internet Of Things/2026/Challenge2_26$
cat CQ2/put_counts.txt
  1 basic
  1 dining_room
  3 dining_room,door
  2 dining_room,temperature
  1 hello_post
  1 living_room,door
  2 main_door
```

Figure 13: Number of times a resource appears in the failed PUT requests

Finally, we compared the counts of unique unsuccessful POST requests (X) and PUT requests (Y) for each resource. Looking at the results, only the `/dining_room` resource received an equal number of unique unsuccessful requests, with exactly $X = 1$ (POST) and $Y = 1$ (PUT). Therefore, the total number of resources satisfying $X = Y$ (where $X > 0$) is 1.

3 CQ3

3.1 CQ3a

Question: *How many separate observe notifications have been sent containing values for the resource `/dining_room/temperature`, ignoring retransmissions?*

Answer: 10

Explanation:

The first thing we need to do to answer the question is to find the observe token for the `/dining_room/temperature` endpoint. To do so we write the following filter: `"coap.code == 1 && coap.opt.observe == 0 && coap.opt.uri_path == "dining_room" && coap.opt.uri_path == "temperature"`. Here below we explain what each field means:

- `coap.code == 1`: filters for the GET method
- `coap.opt.observe == 0`: filters to find messages related to the observe mode
- `coap.opt.uri_path == xxx`: specifies the desired resource

The token is `"89b9"` as we can see from the picture below:

```
alexh@alexh-ThinkPad-T14-Gen-4:~/Desktop/Internet Of Things/2026/Challenge2_26$
tshark -r A.pcapng -Y "coap.opt.observe == 0 && coap.code == 1 && coap.opt.uri_p
ath == \"dining_room\" && coap.opt.uri_path == \"temperature\" -T fields -e coa
p.token > CQ3/observe_token.txt
alexh@alexh-ThinkPad-T14-Gen-4:~/Desktop/Internet Of Things/2026/Challenge2_26$
cat CQ3/observe_token.txt
89b9
```

Figure 14: Token that belongs to the observe registration

After this we look for the messages that have the same token found (`coap.token == 89b9`), have the observe option active (`coap.opt.observe`) and are successful (`coap.code == 69`), using this filter: `"coap.token == 89b9 && coap.opt.observe && coap.code == 69"`. Moreover we extract the MIDs, remove the duplicates (dropping retransmissions) and count the number of unique lines.

```
alexh@alexh-ThinkPad-T14-Gen-4:~/Desktop/Internet Of Things/2026/Challenge2_26$
tshark -r A.pcapng -Y 'coap.token == 89b9 && coap.opt.observe && coap.code == 69'
-T fields -e coap.mid | sort -u | wc -l
11
```

Figure 15: Number of unique notifications

The number of unique lines identified is 11. However the first line detected refers to the ACK packet the server sends to acknowledge the GET request. This means we have to subtract one notification from the previous number, which gets us that the number of separate observe notifications is $11 - 1 = 10$.

3.2 CQ3b

Question: How many of the notifications found in point CQ3a are useless (e.g., a waste of traffic)?

Answer: 5

Explanation:

In our context we can define notifications as useless in two cases:

- The server sends a notification, but the client is dead/disconnected.
- The server sends a notification, but the payload is exactly the same as the previous notification. Since the observe mode should notify the client only when the value changes.

The number of packets that relate to the first definition (packets sent are unreachable), can be simply checked using the Wireshark GUI. The filter applied is the same one used for the final part of CQ3a, and gives us these results:

No.	Time	Source	Destination	Protocol	Length	Info
4344	214.777777148	127.0.0.1	127.0.0.1	CoAP/J...	117	ACK, MID:10888, 2.05 Content, JSON (application/json), TKN:89 b9, End of Block #0, /dining_room/temperature
4620	222.532566824	127.0.0.1	127.0.0.1	CoAP/J...	115	CON, MID:35, 2.05 Content, JSON (application/json), TKN:89 b9, /dining_room/temperature
5098	249.484821853	127.0.0.1	127.0.0.1	CoAP/J...	115	CON, MID:38, 2.05 Content, JSON (application/json), TKN:89 b9, /dining_room/temperature
5623	273.825366943	127.0.0.1	127.0.0.1	CoAP/J...	115	CON, MID:43, 2.05 Content, JSON (application/json), TKN:89 b9, /dining_room/temperature
8509	308.820865762	127.0.0.1	127.0.0.1	CoAP/J...	115	CON, MID:45, 2.05 Content, JSON (application/json), TKN:89 b9, /dining_room/temperature
9414	326.429577951	127.0.0.1	127.0.0.1	CoAP/J...	115	CON, MID:49, 2.05 Content, JSON (application/json), TKN:89 b9, /dining_room/temperature
9866	350.769651227	127.0.0.1	127.0.0.1	CoAP/J...	115	CON, MID:54, 2.05 Content, JSON (application/json), TKN:89 b9, /dining_room/temperature
9867	350.769668653	127.0.0.1	127.0.0.1	ICMP	143	Destination unreachable (Port unreachable)
9937	353.609616139	127.0.0.1	127.0.0.1	CoAP/J...	115	CON, MID:54, 2.05 Content, JSON (application/json), TKN:89 b9, /dining_room/temperature [Retransmission]
9938	353.609648070	127.0.0.1	127.0.0.1	ICMP	143	Destination unreachable (Port unreachable)
10001	359.552073924	127.0.0.1	127.0.0.1	CoAP/J...	115	CON, MID:54, 2.05 Content, JSON (application/json), TKN:89 b9, /dining_room/temperature [Retransmission]
10002	359.552088425	127.0.0.1	127.0.0.1	ICMP	143	Destination unreachable (Port unreachable)
10193	371.257323429	127.0.0.1	127.0.0.1	CoAP/J...	115	CON, MID:54, 2.05 Content, JSON (application/json), TKN:89 b9, /dining_room/temperature [Retransmission]
10194	371.257333119	127.0.0.1	127.0.0.1	ICMP	143	Destination unreachable (Port unreachable)
10573	394.669233647	127.0.0.1	127.0.0.1	CoAP/J...	115	CON, MID:62, 2.05 Content, JSON (application/json), TKN:89 b9, /dining_room/temperature
10574	394.669244742	127.0.0.1	127.0.0.1	ICMP	143	Destination unreachable (Port unreachable)
10631	397.475501427	127.0.0.1	127.0.0.1	CoAP/J...	115	CON, MID:62, 2.05 Content, JSON (application/json), TKN:89 b9, /dining_room/temperature [Retransmission]
10632	397.475510907	127.0.0.1	127.0.0.1	ICMP	143	Destination unreachable (Port unreachable)
11220	436.761538592	127.0.0.1	127.0.0.1	CoAP/J...	115	CON, MID:74, 2.05 Content, JSON (application/json), TKN:89 b9, /dining_room/temperature
11221	436.761561322	127.0.0.1	127.0.0.1	ICMP	143	Destination unreachable (Port unreachable)
11917	467.521064665	127.0.0.1	127.0.0.1	CoAP/J...	115	CON, MID:87, 2.05 Content, JSON (application/json), TKN:89 b9, /dining_room/temperature
11918	467.521076110	127.0.0.1	127.0.0.1	ICMP	143	Destination unreachable (Port unreachable)
12782	498.166885839	127.0.0.1	127.0.0.1	CoAP/J...	115	CON, MID:90, 2.05 Content, JSON (application/json), TKN:89 b9, /dining_room/temperature
12783	498.166901834	127.0.0.1	127.0.0.1	ICMP	143	Destination unreachable (Port unreachable)

Figure 16: WS GUI that shows unreachable packets and retransmissions

By looking at the result, we can clearly distinguish the previously mentioned 10 packets. These are the ones in blue, excluding the retransmissions and the ACK. Moreover, the unreachable packets we count from the images are 5, having MIDs = {54, 62, 74, 87, 90}.

For the second definition (payload being the same as the previous notification) we need to extract the payload from the remaining packets having MIDs = {35, 38, 43, 45, 49}, and look if it changes between transmissions. To do that we can simply use the Wireshark GUI once again and expand the "JavaScript Object Notation" which contains the values for each one of our packets. Below the list of all the values we extracted.

- MID = 35 → 22.32
- MID = 38 → 18.85
- MID = 43 → 25.98
- MID = 45 → 15.59

- $MID = 49 \rightarrow 25.35$

No subsequent notifications contain the same payload, which means that the number of useless notifications is composed only by the unreachable messages found before, which is 5.

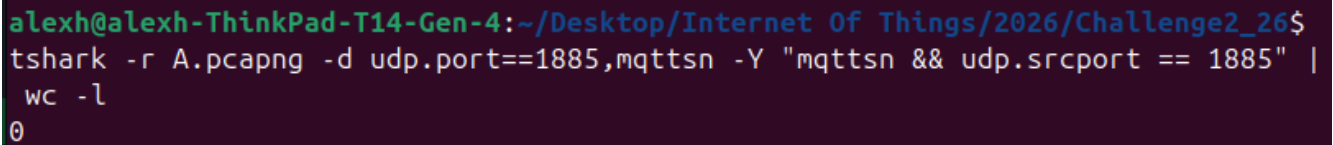
4 CQ4

Question: *How many MQTT-SN messages are received by the clients from the local broker listening on port 1885?*

Answer: 0

Explanation:

To answer this question, we use this command line `"-d udp.port==1885,mqtt-sn"` which forces tshark to decode all UDP traffic on port 1885 as MQTT-SN protocol data. After that we apply the filter `"mqtt_sn && udp.srcport == 1885"` which ensures we are only looking at valid MQTT-SN packets (`mqtt_sn`) and filters for messages originating from the broker and heading to the clients (`udp.srcport == 1885`). Ultimately we count all the lines originating from this filtering.



```
alexh@alexh-ThinkPad-T14-Gen-4:~/Desktop/Internet Of Things/2026/Challenge2_26$
tshark -r A.pcapng -d udp.port==1885,mqttsn -Y "mqttsn && udp.srcport == 1885" |
wc -l
0
```

Figure 17: Number of mqtt-sn messages

5 CQ5

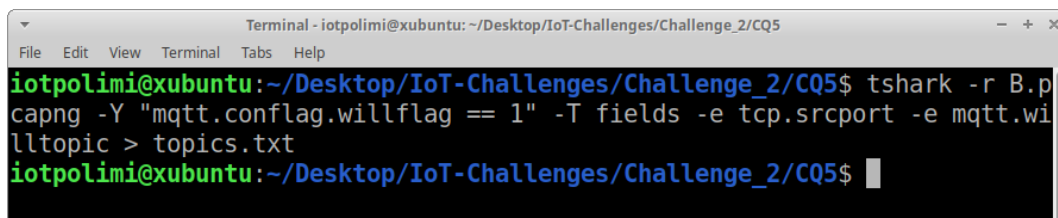
Question: *How many MQTT subscribers receive a last will message derived from a subscription that includes at least one wildcard?*

Answer: 1

Explanation:

To answer this question, we investigated which MQTT subscribers received a Last Will Message (LWM) caused by a wildcard subscription. We achieved this by first identifying the LWM topics and the crashed clients, correlating them to find the exact topic where the failure notification was published. Finally, we verified which of the receiving clients had an active wildcard subscription encompassing that specific path.

1. **Last Will Topics Collection:** We parsed the traffic to collect the unique topics containing a 'Last Will' flag, identifying the configured topic for each client:



```
iotpolimi@xubuntu:~/Desktop/IoT-Challenges/Challenge_2/CQ5$ tshark -r B.p
capng -Y "mqtt.conflag.willflag == 1" -T fields -e tcp.srcport -e mqtt.wi
lltopic > topics.txt
iotpolimi@xubuntu:~/Desktop/IoT-Challenges/Challenge_2/CQ5$
```

Output

```
38083 university/department12/room1/temperature
56285 metaverse/room2/floor4
53485 hospital/facility3/area3
42665 metaverse/room2/room2
```

2. **Crashed Clients Analysis:** We analyzed the TCP reset packets (`tcp.flags.reset == 1`) to determine the presence of LWM configurations on the originating client side, isolating clients that disconnected abruptly:

```
Terminal - iotpolimi@xubuntu: ~/Desktop/IoT-Challenges/Challenge_2/CQ5
File Edit View Terminal Tabs Help
iotpolimi@xubuntu:~/Desktop/IoT-Challenges/Challenge_2/CQ5$ cut -f1 topics
s.txt | xargs -I{} tshark -r B.pcapng -Y "tcp.flags.reset == 1 && tcp.src
port == {}" -T fields -e tcp.srcport -e tcp.dstport > crash_clients.txt
iotpolimi@xubuntu:~/Desktop/IoT-Challenges/Challenge_2/CQ5$
```

Output

```
38083 1883
```

3. **Failure Correlation:** We correlated the Last Will Topics with the failing clients to establish the single topic of interest where the broker dispatched the LWM:

```
Terminal - iotpolimi@xubuntu: ~/Desktop/IoT-Challenges/Challenge_2/CQ5
File Edit View Terminal Tabs Help
iotpolimi@xubuntu:~/Desktop/IoT-Challenges/Challenge_2/CQ5$ grep "^$(cut
-f1 ./crash_clients.txt)" ./topics.txt | cut -f2 > topics_with_failures.t
xt
iotpolimi@xubuntu:~/Desktop/IoT-Challenges/Challenge_2/CQ5$
```

Output

```
university/department12/room1/temperature
```

4. **Broker Messages Destinations Retrieval:** Having established the topic, we retrieved the messages dispatched by the broker within that topic and isolated their destination addresses (the receiving clients):

```
Terminal - iotpolimi@xubuntu: ~/Desktop/IoT-Challenges/Challenge_2/CQ5
File Edit View Terminal Tabs Help
iotpolimi@xubuntu:~/Desktop/IoT-Challenges/Challenge_2/CQ5$ tshark -r B.p
capng -Y "mqtt.conflg.willflag == 1" -T fields -e tcp.srcport -e mqtt.wi
lltopic > topics.txt -r B.pcapng -Y 'tcp.srcport == 1883 && mqtt.msgtype
== 3 && mqtt.topic == "university/department12/room1/temperature"' -T fie
lds -e tcp.dstport > sent_messages_to_clients.txt
iotpolimi@xubuntu:~/Desktop/IoT-Challenges/Challenge_2/CQ5$
```

Output

```
39551
53557
51743
41789
```

5. **Wildcard Subscriptions Verification:** Finally, we checked the active subscriptions containing at least one wildcard (`#` or `+`) and cross-referenced them with the receiving clients. By extracting the specific wildcard topics, we verified that the multi-level wildcard (e.g., `university/#`) logically encompassed the LWM's specific path, conclusively validating it as the correct answer:

```
Terminal - iotpolimi@xubuntu: ~/Desktop/IoT-Challenges/Challenge_2/CQ5
File Edit View Terminal Tabs Help
iotpolimi@xubuntu:~/Desktop/IoT-Challenges/Challenge_2/CQ5$ tshark -r B.pcapng -Y 'mqtt.msgtype == 8 && (mqtt.topic contains "#" || mqtt.topic contains "+")' -T fields -e tcp.srcport -e mqtt.topic | grep -F -f sent_messages_to_clients.txt | sort -u > subscribed_clients.txt
iotpolimi@xubuntu:~/Desktop/IoT-Challenges/Challenge_2/CQ5$
```

Output

```
51743 hospital/+/section0/temperature
51743 house/department3/+
51743 house/department3/+/+
51743 house/+/room1/#
51743 metaverse/+
51743 metaverse/facility0/section0/+
51743 university/#
```

6 CQ6

6.1 CQ6a

Question: *How many different MQTT clients sent at least one message to erase a previously retained value in a topic in the public HiveMQ broker?*

Answer: 5

Explanation:

To answer this question, we first identified the IP addresses of the public HiveMQ broker by analyzing the DNS queries. Since deleting a retained message in the MQTT protocol requires sending a PUBLISH packet with the retain=1 flag and an empty payload, we then filtered the traffic to find these specific events and isolate the target clients.

1. **Broker IPs Identification:** We used tshark to extract the resolved IP addresses for the broker.hivemq.com domain and created a display filter to isolate only the traffic directed to those specific IPs:

```
NODE_ADDRS=$(tshark -r B.pcapng -Y "dns.qry.name == \"broker.hivemq.com\"" -T fields
↪ -e dns.a | tr ',' '\n' | sort -u | awk NF | paste -sd ' ')
```

```
TRAFFIC_RULE="ip.dst in ${NODE_ADDRS}"
```

2. **Deletion Messages Filtering (Retain = 1, Empty Payload):** We filtered the outgoing packets looking for msgtype==3 (PUBLISH) and retain==1. By extracting the payload and the source TCP port (which uniquely identifies the client session), we isolated the connections with an empty payload (represented as <MISSING>) and counted the unique source ports:

```
tshark -r B.pcapng -Y "$TRAFFIC_RULE && mqtt.msgtype==3 && mqtt.retain==1" -T fields
↪ -E separator='|' -e tcp.srcport -e mqtt.msg > publish_events.txt
```

```
awk -F'|' ' $2=="<MISSING>" {print $1}' publish_events.txt | sort -u > active_ports.txt
```

```
RESULT_A=$(wc -l < active_ports.txt)
```

Output

```
34779
36777
38207
47753
49877
```

6.2 CQ6b

Question: *How many of the clients found in CQ6a) connected to the HiveMQ broker with a client identifier strictly longer than 7 bytes (>7)?*

Answer: 2

Explanation:

This part correlates the network sessions found in the previous point with their actual MQTT client identities. We extracted the Client IDs from the MQTT CONNECT messages and joined this mapping with our active ports list to evaluate the string length.

1. **Client IDs Extraction:** We filtered the packet capture for MQTT CONNECT messages (msgtype==1) to extract and map the source TCP ports to their corresponding mqtt.clientid strings:

```
tshark -r B.pcapng -Y "$TRAFFIC_RULE && mqtt.msgtype==1" -T fields -e tcp.srcport -e
↪ mqtt.clientid > connection_map.txt
```

2. **Client Mapping and Length Evaluation:** We joined this mapping with the list of active source ports identified in CQ6a to isolate our target devices. Finally, we evaluated the character length of each ID, keeping only those strictly longer than 7 bytes:

```
grep -F -f active_ports.txt connection_map.txt | sort -u > target_devices.txt

RESULT_B=$(awk '{print $2}' target_devices.txt | awk 'length($0)>7' | wc -l)
```

Output

```
38207 giuxfijwus
47753 rolsmcwhshlbc
```

7 CQ7

Question: *How many MQTT subscribe requests directed to the local broker specify a topic with at least two wildcards?*

Answer: 9

Explanation:

To find the number of requests specifying at least two wildcards, we parsed the network traffic to isolate the relevant MQTT subscription topics. We extracted the traffic targeting the local IPv6 broker and applied a regular expression to filter the topics

1. **Local Subscriptions Extraction:** We used tshark to filter the capture for MQTT subscribe requests (msgtype==8) explicitly directed to the local IPv6 broker (ipv6.dst==::1) and extracted the raw topic strings:

```
!tshark -r B.pcapng -Y "mqtt.msgtype==8 && ipv6.dst==::1" -T fields -e mqtt.topic >
↪ topics_extracted.txt
```

2. **Wildcard Pattern Matching:** We applied a grep regular expression to the extracted list to filter and keep only the topics containing at least two wildcard characters (+ or #). Finally, we counted the remaining lines:

```
!grep -E '(\+|#).*(\+|#)' topics_extracted.txt > multiple_matches.txt
```

Output

```
university/+//deposit
hospital/+//light
metaverse/+//room1/#
house/department3/+//
metaverse/facility0/+//
hospital/fxlugs/+/#
metaverse/+//hydraulic_valve
factory/+/#
house/+//room1/#
```

8 CQ8

Question: Compare the local broker usage in the two capture files *A.pcapng* and *B.pcapng*. The goal is to have a plot on the number of layers of the topics used in the publish messages directed to the local broker in the two captures. Produce a histogram plot with number of messages for each number of layers

Answer:

CQ8a: 439

CQ8b: 534

Explanation:

To answer this question, we first extracted the topics from the PUBLISH messages directed to the local broker for both captures, and then we analyzed the number of layers using a Python script.

1. **Topics Extraction:** We used tshark to filter both packet captures for MQTT PUBLISH messages (`mqtt.msgtype == 3`) explicitly directed to the local broker (`tcp.dstport == 1883`) on either IPv4 or IPv6 localhost addresses (`ip.dst==127.0.0.1 || ipv6.dst== ::1`). We extracted the raw topic strings into separate text files:

```
tshark -r A.pcapng -Y "mqtt.msgtype == 3 and tcp.dstport == 1883 and
↪ (ip.dst==127.0.0.1 || ipv6.dst== ::1)" -T fields -e mqtt.topic > topics_A_raw.txt
tshark -r B.pcapng -Y "mqtt.msgtype == 3 and tcp.dstport == 1883 and
↪ (ip.dst==127.0.0.1 || ipv6.dst== ::1)" -T fields -e mqtt.topic > topics_B_raw.txt
```

2. **Layers Calculation:** We processed the extracted files using an optimized Python script. Since some entries contained multiple topics separated by commas, we split those first. Then, to efficiently calculate the number of layers for each topic, we counted the number of slash separators (/) and added one (`layer_count = topic.count('/') + 1`), avoiding unnecessary string allocations.

```
def extract_and_count_layers(filename):
    try:
        with open(filename, 'r') as f:
            return [
                topic.strip().count('/') + 1
                for line in f
                for topic in line.split(',')
                if topic.strip()
            ]
    except FileNotFoundError:
        print(f"File {filename} not found")
        return []
```

3. **Distribution Analysis:** We mapped the layer counts to their frequencies using the `collections.Counter` module to obtain the final distribution for both captures.

Output

```
Distribution A: 2: 154, 3: 131, 4: 154  
Distribution B: 1: 21, 2: 172, 3: 162, 4: 179
```

4. **Plot Generation:** Finally, we used the `matplotlib.pyplot` library to produce an histogram comparing the distribution of layers between A.pcapng and B.pcapng. The generated plot correctly displays the "Number of topic layers" on the X-axis and the "Number of publish messages" on the Y-axis.

